

Laboratorium nr. 8, laboratorium nr. 9, laboratorium nr. 10	
Imię i nazwisko: Miłosz Dębowski	Kierunek: Informatyka Techniczna
Numer albumu: 415045	Gr. Lab.: 8

Laboratorium nr. 8

Cel: doskonalenie umiejętności realizacji synchronizacji w języku C za pomocą zmiennych warunku oraz w programach obiektowych w Javie za pomocą narzędzi pakietu `java.util.concurrent`

Bariera

1. Przygotowanie środowiska:

- Utworzenie katalogów roboczych: `lab_8` oraz podkatalogów (`lab_8_bariera` i `lab_8_threads`).
- Skopiowanie odpowiednich plików źródłowych.

2. Implementacja funkcji bariery:

- Utworzenie zmiennych globalnych do zliczania wątków oraz synchronizacji.

```
static int ilosc_watkow = 0;
static int ilosc_watkow_w_funkcji = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condition = PTHREAD_COND_INITIALIZER;
```

- Implementacja funkcji `bariera_init` i `bariera`.

```
void bariera_init(int n) {
    ilosc_watkow = n;
}
void bariera() {
    pthread_mutex_lock(&lock);

    ilosc_watkow_w_funkcji++;

    if (ilosc_watkow_w_funkcji == ilosc_watkow) {
        ilosc_watkow_w_funkcji = 0;
        pthread_cond_broadcast(&condition);
    } else {
        pthread_cond_wait(&condition, &lock);
    }

    pthread_mutex_unlock(&lock);
}
```

3. Opis działania kodu

- Funkcja `bariera_init()` ustawia liczbę wątków, które mają dojść do bariery.
- Wątki wchodzi do bariery i blokują mutex (`pthread_mutex_lock`).
- Licznik `ilosc_watkow_w_funkcji` jest zwiększany o 1.
- Jeśli liczba wątków osiągnie wartość `ilosc_watkow`, następuje odblokowanie wszystkich wątków za pomocą `pthread_cond_broadcast`, a licznik zostaje zresetowany do 0.
- Wątki, które nie osiągnęły jeszcze bariery, czekają na sygnał warunkowy (`pthread_cond_wait`).
- Mutex jest odblokowywany na zakończenie.

4. Testowanie i wyniki

Testy przeprowadzono dla 4 wątków. Każdy wątek wyświetla komunikat przed oraz po dotarciu do bariery. Wyniki zostały przedstawione na poniższym zrzucie ekranu:

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_8/lab_8_bariera$ ./main
przed bariera 1 - watek 0
przed bariera 1 - watek 1
przed bariera 1 - watek 2
przed bariera 1 - watek 3
przed bariera 2 - watek 1
przed bariera 2 - watek 0
przed bariera 2 - watek 3
przed bariera 2 - watek 2
przed bariera 3 - watek 1
przed bariera 3 - watek 0
przed bariera 3 - watek 2
przed bariera 3 - watek 3
przed bariera 4 - watek 0
przed bariera 4 - watek 1
przed bariera 4 - watek 2
przed bariera 4 - watek 3
po ostatniej barierze - watek 0
po ostatniej barierze - watek 1
po ostatniej barierze - watek 3
po ostatniej barierze - watek 2
```

5. Analiza wyników

- Wątki są zsynchronizowane przy każdej barierze. Żaden z wątków nie kontynuuje pracy przed osiągnięciem bariery przez wszystkie pozostałe.
- Kolejne bariery są osiągnęte zgodnie z założeniami, a licznik `ilosc_watkow_w_funkcji` jest poprawnie resetowany.
- Kolejność wątków jest nieokreślona, co jest naturalnym zachowaniem w programowaniu równoległym.

6. Wnioski

- Implementacja bariery za pomocą mutexów i zmiennych warunkowych umożliwia skuteczną synchronizację wątków.
- Wykorzystanie funkcji `pthread_cond_broadcast` pozwala na odblokowanie wszystkich oczekujących wątków jednocześnie.
- Testy potwierdziły poprawność działania kodu – żaden z wątków nie ruszył dalej, dopóki wszystkie nie osiągnęły bariery.

Czytelnia

1. Przygotowanie środowiska

- Utworzenie katalogu roboczego (np. `lab_8_pthreads`).
- Rozpakowanie paczki `CzytPis_Pthreads.tgz` i zapoznanie się z dostarczonym kodem.
- Kompilacja i testowanie kodu bazowego w celu weryfikacji jego działania.

2. Analiza kodu i pseudokodu

- Przeanalizowanie pseudokodu monitora Czytelnia z wykładu 6.
- Porównanie pseudokodu z funkcjami zaimplementowanymi w pliku `czytelnia.c` (`my_read_lock_lock`, `my_read_lock_unlock`, `my_write_lock_lock`, `my_write_lock_unlock`).

3. Implementacja protokołu synchronizacji

- W funkcji `my_read_lock_lock`:
 - Sprawdzenie, czy pisarze są aktywni lub czekają.
 - Jeśli tak, zwiększenie liczby oczekujących czytelników (`l_c_czekajacych`) i oczekiwanie na sygnał od pisarzy.
 - Po wejściu do sekcji krytycznej zwiększenie liczby aktywnych czytelników (`l_c`).

```
int my_read_lock_lock(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    if(czytelnia_p->l_p > 0 || l_p_czekajacych > 0){
        l_c_czekajacych++;
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock_czytelnicy);
        pthread_cond_wait(&cond_czytelnicy, &lock_czytelnicy);
        pthread_mutex_unlock(&lock_czytelnicy);
        pthread_mutex_lock(&lock);
        l_c_czekajacych--;
    }
    czytelnia_p->l_c++;
    pthread_cond_signal (&cond_czytelnicy);
    pthread_mutex_unlock (&lock);
}
```

- W funkcji `my_read_lock_unlock`:
 - Zmniejszenie liczby aktywnych czytelników.
 - Wysyłanie sygnału do pisarzy, jeśli żaden czytelnik nie pozostał w czytelni.

```
int my_read_lock_unlock(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    czytelnia_p->l_c--;
    if(czytelnia_p->l_c ==0){
        pthread_cond_signal(&cond_pisarze);
    }
    pthread_mutex_unlock (&lock);
}
```

- W funkcji `my_write_lock_lock`:
 - Sprawdzenie, czy inni czytelnicy lub pisarze są aktywni.
 - Jeśli tak, zwiększenie liczby oczekujących pisarzy (`l_p_czekajacych`) i oczekiwanie na sygnał od czytelników.

- Po wejściu do sekcji krytycznej zwiększenie liczby aktywnych pisarzy (`l_p`).

```
int my_write_lock_lock(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    if(czytelnia_p->l_c + czytelnia_p->l_p > 0) {
        l_p_czekajacych++;
        pthread_mutex_unlock (&lock);
        pthread_mutex_lock(&lock_pisarze);
        pthread_cond_wait(&cond_pisarze, &lock_pisarze);
        pthread_mutex_unlock(&lock_pisarze);
        pthread_mutex_lock(&lock);
        l_p_czekajacych--;
    }
    czytelnia_p->l_p++;
    pthread_mutex_unlock (&lock);
}
```

- W funkcji `my_write_lock_unlock`:
 - Zmniejszenie liczby aktywnych pisarzy.
 - Wysyłanie sygnału do czytelników lub pisarzy w zależności od sytuacji.

```
int my_write_lock_unlock(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    czytelnia_p->l_p--;
    if(l_c_czekajacych > 0){
        pthread_cond_signal(&cond_czytelnicy);
    }
    else {
        pthread_cond_signal(&cond_pisarze);
    }
    pthread_mutex_unlock (&lock);
}
```

4. Obsługa błędów i debugowanie

- Dodanie wydruków kontrolnych w funkcjach `czytam` i `pisze`, które wyświetlają aktualne wartości zmiennych (`l_c`, `l_p`, `l_c_czekajacych`, `l_p_czekajacych`).
- Sprawdzenie warunków poprawności synchronizacji, np. czy liczby czytelników i pisarzy nie są ujemne lub czy nie dochodzi do jednoczesnego odczytu i zapisu.

- Włączenie sekcji debugowania przy pomocy kompilacji warunkowej (#ifdef MY_DEBUG).

```
void czytam(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    #ifdef MY_DEBUG
        printf("liczba czytelników = %d, liczba pisarzy = %d\n",
            czytelnia_p->l_c, czytelnia_p->l_p, l_c_czekajacych, l_p_czekajacych);
    #endif
    if(czytelnia_p->l_p>1 || (czytelnia_p->l_p==1 && czytelnia_p->l_c>0) ||
        czytelnia_p->l_p<0 || czytelnia_p->l_c<0 ) {
        #ifdef MY_DEBUG
            printf("Nie powinienem czytać\n");
        #endif
        pthread_mutex_unlock (&lock);
        exit(0);
    }
    pthread_mutex_unlock (&lock);
    usleep(rand()%300000);
}

void pisze(czytelnia_t* czytelnia_p){
    pthread_mutex_lock (&lock);
    #ifdef MY_DEBUG
        printf("liczba czytelników = %d, liczba pisarzy = %d\n",
            czytelnia_p->l_c, czytelnia_p->l_p, l_c_czekajacych, l_p_czekajacych);
    #endif
    if( czytelnia_p->l_p>1 || (czytelnia_p->l_p==1 && czytelnia_p->l_c>0) ||
        czytelnia_p->l_p<0 || czytelnia_p->l_c<0 ) {
        #ifdef MY_DEBUG
            printf("Nie powinienem pisać\n");
        #endif
        pthread_mutex_unlock (&lock);
        exit(0);
    }
    pthread_mutex_unlock (&lock);
    usleep(rand()%300000);
}
```

5. Wyniki

- Logi z testów, które potwierdzają prawidłowe kolejkowanie wątków czytelników i pisarzy:
 - Przykładowe fragmenty logów z wydrukami MY_DEBUG.

```
czytelnik 139764701382336 - wychodze
czytelnik 139764701382336 - po zamku
czytelnik 139764709775040 - przed zamkiem
czytelnik 139764709775040 - wchodzi
liczba czytelnikow = 2, liczba pisarzy = 0
liczba oczekujących czytelników = 0, liczba oczekujących pisarzy = 0
czytelnik 139764726560448 - wychodze
czytelnik 139764726560448 - po zamku
czytelnik 139764734953152 - przed zamkiem
czytelnik 139764734953152 - wchodzi
liczba czytelnikow = 2, liczba pisarzy = 0
liczba oczekujących czytelników = 0, liczba oczekujących pisarzy = 0
czytelnik 139764709775040 - wychodze
czytelnik 139764709775040 - po zamku
czytelnik 139764751738560 - przed zamkiem
czytelnik 139764751738560 - wchodzi
liczba czytelnikow = 2, liczba pisarzy = 0
liczba oczekujących czytelników = 0, liczba oczekujących pisarzy = 0
czytelnik 139764743345856 - przed zamkiem
czytelnik 139764743345856 - wchodzi
liczba czytelnikow = 3, liczba pisarzy = 0
liczba oczekujących czytelników = 0, liczba oczekujących pisarzy = 0
czytelnik 139764676204224 - przed zamkiem
czytelnik 139764676204224 - wchodzi
liczba czytelnikow = 4, liczba pisarzy = 0
liczba oczekujących czytelników = 0, liczba oczekujących pisarzy = 0
czytelnik 139764743345856 - wychodze
czytelnik 139764743345856 - po zamku
pisarz 139764785309376 - przed zamkiem
czytelnik 139764734953152 - wychodze
czytelnik 139764734953152 - po zamku
czytelnik 139764751738560 - wychodze
czytelnik 139764751738560 - po zamku
czytelnik 139764751738560 - przed zamkiem
czytelnik 139764684596928 - przed zamkiem
czytelnik 139764676204224 - wychodze
czytelnik 139764676204224 - po zamku
pisarz 139764785309376 - wchodzi
liczba czytelnikow = 0, liczba pisarzy = 1
liczba oczekujących czytelników = 2, liczba oczekujących pisarzy = 0
pisarz 139764793702080 - przed zamkiem
czytelnik 139764676204224 - przed zamkiem
pisarz 139764785309376 - wychodze
pisarz 139764785309376 - po zamku
czytelnik 139764751738560 - wchodzi
czytelnik 139764676204224 - wchodzi
```

6. Wnioski

- Implementacja problemu czytelników i pisarzy została poprawnie zrealizowana z użyciem mechanizmów pthreads.
- Udało się osiągnąć wzajemne wykluczanie oraz właściwe kolejkovanie wątków za pomocą mutexów i zmiennych warunkowych.
- Działanie programu zostało przetestowane w różnych konfiguracjach i nie wykryto błędów w synchronizacji.
- Mechanizm debugowania (MY_DEBUG) okazał się pomocny w wykrywaniu problemów na wczesnym etapie implementacji.
- Użycie zmiennych warunkowych minimalizuje aktywne oczekiwanie wątków, co skutkuje mniejszym obciążeniem procesora.
- Implementacja priorytetów (kolejkowanie oczekujących czytelników i pisarzy) zapewnia sprawiedliwość dostępu do zasobów.

Laboratorium nr. 9

Cel: Nabycie umiejętności tworzenia i implementacji programów równoległych z wykorzystaniem OpenMP.

openmp_petle_simple

1. Analiza kodu i uruchomienie programu openmp_petle_simple.c

- Kod źródłowy openmp_petle_simple.c zawiera prostą implementację obliczania sumy elementów tablicy w sposób sekwencyjny oraz w pętli przygotowanej do zrównoleglenia.

Kompilacja programu wykonana została poleceniem:

```
gcc -fopenmp openmp_petle_simple.c -o openmp_petle_simple
```

- Uruchomienie programu wykazało poprawność sumowania w wersji sekwencyjnej. Wynik sumy to 153.00 (wartość przybliżona).

2. Zrównoleglenie pętli i testowanie z czterema wątkami

- Pętla została zrównoleglona z użyciem dyrektywy `#pragma omp parallel for`. Dodano:
 - Klauzulę `default(none)` dla jawnego zdefiniowania zmiennych.
 - Klauzulę `reduction(+:suma_parallel)` dla redukcji.
- Kod zmodyfikowany:

```
#pragma omp parallel for default(none) reduction(+:suma_parallel) shared(a)
ordered
for(int i=0;i<WYMIAR;i++) {
    int id_w = omp_get_thread_num();
    suma_parallel += a[i];

    #pragma omp ordered
    printf("a[%2d]->W_%1d  \n",i,id_w);
}
```

- Obserwacje:
 - Wynik sumowania równoległego jest zgodny z sekwencyjnym.
 - Dodanie klauzuli `ordered` wymusza wydruk w kolejności iteracji.

```
Suma wyrazów tablicy: 156.060000
a[ 0]->W_0
a[ 1]->W_0
a[ 2]->W_0
a[ 3]->W_0
a[ 4]->W_0
a[ 5]->W_1
a[ 6]->W_1
a[ 7]->W_1
a[ 8]->W_1
a[ 9]->W_1
a[10]->W_2
a[11]->W_2
a[12]->W_2
a[13]->W_2
a[14]->W_3
a[15]->W_3
a[16]->W_3
a[17]->W_3

Suma wyrazów tablicy równolegle (z klauzulą - ....: 156.060000
```

- Wynik dla czterech wątków
 - Wątki równoległe przydzieliły iteracje, a ich podział był zgodny z oczekiwaniami.
- 3. Testy z klauzulą schedule
 - Przetestowano cztery warianty klauzuli schedule z czterema wątkami:

Static, rozmiar porcji = 3:

```
#pragma omp parallel for default(none) schedule(static, 3) num_threads(4)
reduction(+:suma_parallel) shared(a) ordered
```

Obserwacje: Iteracje były przydzielane porcjami po 3 do wątków, w sposób cykliczny.

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_9$ ./openmp_petle_simple
Suma wyrazów tablicy: 156.060000
a[ 0]->W_0
a[ 1]->W_0
a[ 2]->W_0
a[ 3]->W_1
a[ 4]->W_1
a[ 5]->W_1
a[ 6]->W_2
a[ 7]->W_2
a[ 8]->W_2
a[ 9]->W_3
a[10]->W_3
a[11]->W_3
a[12]->W_0
a[13]->W_0
a[14]->W_0
a[15]->W_1
a[16]->W_1
a[17]->W_1

Suma wyrazów tablicy równoległe (z klauzulą - ....: 156.060000
```

Static, rozmiar porcji domyślny:

```
#pragma omp parallel for default(none) schedule(static) num_threads(4)
reduction(+:suma_parallel) shared(a) ordered
```

Obserwacje: Domyślny rozmiar porcji to 5 (WYMIAR/4).

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_9$ ./openmp_petle_simple
Suma wyrazów tablicy: 156.060000
a[ 0]->W_0
a[ 1]->W_0
a[ 2]->W_0
a[ 3]->W_0
a[ 4]->W_0
a[ 5]->W_1
a[ 6]->W_1
a[ 7]->W_1
a[ 8]->W_1
a[ 9]->W_1
a[10]->W_2
a[11]->W_2
a[12]->W_2
a[13]->W_2
a[14]->W_3
a[15]->W_3
a[16]->W_3
a[17]->W_3

Suma wyrazów tablicy równoległe (z klauzulą - ....: 156.060000
```

Dynamic, rozmiar porcji = 2:

```
#pragma omp parallel for default(none) schedule(dynamic, 2) num_threads(4)
reduction(+:suma_parallel) shared(a) ordered
```

Obserwacje: Iteracje były przydzielane w sposób dynamiczny, porcjami po 2. Kolejność przydzielania iteracji mogła się zmieniać.

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_9$ ./openmp_petle_simple
Suma wyrazów tablicy: 156.060000
a[ 0]->W_1
a[ 1]->W_1
a[ 2]->W_0
a[ 3]->W_0
a[ 4]->W_2
a[ 5]->W_2
a[ 6]->W_3
a[ 7]->W_3
a[ 8]->W_1
a[ 9]->W_1
a[10]->W_0
a[11]->W_0
a[12]->W_2
a[13]->W_2
a[14]->W_3
a[15]->W_3
a[16]->W_1
a[17]->W_1
Suma wyrazów tablicy równoległe (z klauzulą - ....: 156.060000
```

Dynamic, rozmiar porcji domyślny:

```
#pragma omp parallel for default(none) schedule(dynamic) num_threads(4)
reduction(+:suma_parallel) shared(a) ordered
```

Obserwacje: Domyślny rozmiar porcji to 1. Iteracje były dynamicznie przydzielane, co umożliwiło większą równoległość.

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_9$ ./openmp_petle_simple
Suma wyrazów tablicy: 156.060000
a[ 0]->W_2
a[ 1]->W_0
a[ 2]->W_3
a[ 3]->W_1
a[ 4]->W_2
a[ 5]->W_0
a[ 6]->W_3
a[ 7]->W_1
a[ 8]->W_2
a[ 9]->W_0
a[10]->W_3
a[11]->W_1
a[12]->W_2
a[13]->W_0
a[14]->W_3
a[15]->W_1
a[16]->W_2
a[17]->W_0
Suma wyrazów tablicy równoległe (z klauzulą - ....: 156.060000
```

4. Wnioski

- Dyrektywa bez klauzuli schedule domyślnie stosuje schedule(static) z domyślnym rozmiarem porcji.
- W przypadku dynamic, podział iteracji może różnić się między uruchomieniami.

openmp_petle

1. Funkcja tablica:

- Inicjalizuje tablicę o wymiarach 10x10

```
void tablica(double a[WYMIAR][WYMIAR]) {
    for (int i = 0; i < WYMIAR; i++) {
        for (int j = 0; j < WYMIAR; j++) {
            a[i][j] = 1.02 * i + 1.01 * j;
        }
    }
}
```

2. Funkcja suma_sekwencyjnie:

- Oblicza sumę elementów tablicy w sposób sekwencyjny.

```
double suma_sekwencyjnie(double a[WYMIAR][WYMIAR]) {  
    double suma = 0.0;  
    for (int i = 0; i < WYMIAR; i++) {  
        for (int j = 0; j < WYMIAR; j++) {  
            suma += a[i][j];  
        }  
    }  
    return suma;  
}
```

3. Funkcja suma_rownolegle_wiersze:

- Implementuje równoległą dekompozycję wierszową, używając dyrektywy #pragma omp parallel for z harmonogramem statycznym (rozmiar porcji: 2).
- Użyto dyrektywy ordered do zachowania kolejności wyświetlania wyników.

```
double suma_rownolegle_wiersze(double a[WYMIAR][WYMIAR]) {  
    double suma = 0.0;  
    #pragma omp parallel for default(none) schedule(static, 2)  
    reduction(+:suma) shared(a) ordered  
    for (int i = 0; i < WYMIAR; i++) {  
        for (int j = 0; j < WYMIAR; j++) {  
            #pragma omp ordered  
            {  
                suma += a[i][j];  
                printf("(%1d,%1d) -> W_%1d ", i, j, omp_get_thread_num());  
            }  
        }  
        #pragma omp ordered  
        {  
            printf("\n");  
        }  
    }  
    return suma;  
}
```

4. Funkcja suma_rownolegle_kolumny_wew:

- Implementuje równoległą dekompozycję kolumnową, z wykorzystaniem zewnętrznej pętli iterującej po wierszach, podczas gdy pętla wewnętrzna jest zrównolegleniona.

```
double suma_rownolegle_kolumny_wew(double a[WYMIAR][WYMIAR]) {
    double suma = 0.0;
    for (int i = 0; i < WYMIAR; i++) {
        #pragma omp parallel for default(none) schedule(static)
        reduction(+:suma) shared(a, i) ordered
        for (int j = 0; j < WYMIAR; j++) {
            #pragma omp ordered
            {
                suma += a[i][j];
                printf("(%1d,%1d) -> W_%1d ", i, j, omp_get_thread_num());
            }
        }
        printf("\n");
    }
    return suma;
}
```

5. Funkcja suma_rownolegle_kolumny_zew:

- Implementuje równoległą dekompozycję kolumnową, z wykorzystaniem zrównoleglenia pętli zewnętrznej. Wykorzystano zmienne prywatne (suma_tmp) oraz sekcję krytyczną do sumowania wyników.

```
double suma_rownolegle_kolumny_zew(double a[WYMIAR][WYMIAR]) {
    double suma = 0.0;
    #pragma omp parallel default(none) shared(a, suma)
    {
        double suma_tmp = 0.0;
        #pragma omp for schedule(static) ordered
        for (int j = 0; j < WYMIAR; j++) {
            #pragma omp ordered
            {
                for (int i = 0; i < WYMIAR; i++) {
                    suma_tmp += a[i][j];
                    printf("(%1d,%1d) -> W_%1d ", i, j, omp_get_thread_num());
                }
                printf("\n");
            }
        }
        #pragma omp critical
        suma += suma_tmp;
    }
    return suma;
}
```

6. Funkcja main:

- Wywołuje opisane powyżej metody i wyświetla wyniki.
- Liczba wątków ustawiona na 3 za pomocą funkcji `omp_set_num_threads`.

```
int main() {
    omp_set_num_threads(3);
    double a[WYMIAR][WYMIAR];
    tablica(a);
    // Sekwencyjne obliczanie sumy
    double suma = suma_sekwencyjnie(a);
    printf("Suma wyrazów tablicy (sekwencyjnie): %lf\n", suma);
    // Dekompozycja wierszowa schedule static, rozmiar porcji=2
    double suma_rownolegle_wiersze_wynik = suma_rownolegle_wiersze(a);
    printf("Suma (wierszowa): %lf\n", suma_rownolegle_wiersze_wynik);
    // Dekompozycja kolumnowa - zrównoleglenie pętli wewnętrznej schedule
dynamic, rozmiar porcji domyślny
    double suma_rownolegle_kolumny_wew_wynik = suma_rownolegle_kolumny_wew(a);
    printf("Suma (kolumnowa - wewnętrzna): %lf\n",
suma_rownolegle_kolumny_wew_wynik);
    // Dekompozycja kolumnowa - zrównoleglenie pętli zewnętrznej
    double suma_rownolegle_kolumny_zew_wynik = suma_rownolegle_kolumny_zew(a);
    printf("Suma (kolumnowa - zewnętrzna): %lf\n",
suma_rownolegle_kolumny_zew_wynik);
    return 0;
}
```

7. Wyniki

- Suma sekwencyjna
 - Wynik: 913.500000
- Suma równoległa (dekompozycja wierszowa, schedule static, rozmiar porcji: 2):
 - Wynik: 913.500000
 - Rozdział pracy pomiędzy wątki:
 - Wątek 0: wiersze 0, 1, 6, 7.
 - Wątek 1: wiersze 2, 3, 8, 9.
 - Wątek 2: wiersze 4, 5.
- Suma równoległa (dekompozycja kolumnowa, zewnętrzna pętla iterująca po wierszach, pętla wewnętrzna zrównolegleniona):
 - Wynik: 913.500000
 - Rozdział pracy pomiędzy wątki:
 - Wątek 0: kolumny 0-3.
 - Wątek 1: kolumny 4-6.

- Wątek 2: kolumny 7-9.
- Suma równoległa (dekompozycja kolumnowa, zrównoleglenie pętli zewnętrznej):
 - Wynik: 913.500000
 - Rozdział pracy pomiędzy wątki:
 - Wątek 0: kolumny 0-3.
 - Wątek 1: kolumny 4-6.
 - Wątek 2: kolumny 7-9.

8. Uwagi dotyczące wyników:

- Wyniki sumowania są identyczne dla wszystkich metod, co świadczy o poprawności implementacji.
- Wyjście konsoli ukazuje poprawny podział pracy pomiędzy wątki.

9. Wnioski

- Metody dekompozycji wierszowej i kolumnowej różnią się rozdziałem pracy pomiędzy wątki, co może mieć wpływ na równomierność obciążenia przy większych tablicach.
- Przy większych tablicach oczekuje się bardziej zauważalnych różnic w czasie wykonania pomiędzy podejściem sekwencyjnym i równoległym. W takim przypadku liczba wątków i strategia harmonogramowania będą kluczowe dla osiągnięcia wysokiej wydajności.
- Eksperyment pokazał, jak różne podejścia do dekompozycji wpływają na wykonywanie obliczeń w środowisku wielowątkowym.

Laboratorium nr. 10

Cel: Nabycie umiejętności tworzenia i implementacji programów równoległych z wykorzystaniem OpenMP.

Zmienne

1. Przygotowanie środowiska:

- Utworzono katalog roboczy lab_10 z podkatalogiem zmienne.
- Skopiowano i rozpakowano paczkę openmp_watki_zmienne.tgz, a następnie skompilowano i uruchomiono program.

2. Analiza i modyfikacja programu:

- Wprowadzono modyfikacje w kodzie:
 - Dodano ochronę sekcji krytycznych dla zmiennej a_shared za pomocą dyrektywy #pragma omp critical.
 - Zastosowano operacje atomowe dla zmiennej e_atomic przy pomocy dyrektywy #pragma omp atomic.
 - Wprowadzono zmienną f_threadprivate, objętą dyrektywą #pragma omp threadprivate, której wartość była ustawiana w pierwszym obszarze równoległym i wykorzystywana w drugim.

```
#include<stdlib.h>
#include<stdio.h>
#include<omp.h>
int f_threadprivate;
#pragma omp threadprivate(f_threadprivate)
int main(){
    #ifdef _OPENMP
        printf("\nKompilator rozpoznaje dyrektywy OpenMP\n");
    #endif
    int liczba_watkow;
    int a_shared = 1;
    int b_private = 2;
    int c_firstprivate = 3;
    int e_atomic=5;
    printf("przed wejściem do obszaru równoległego - nr_threads %d, thread ID %d\n", omp_get_num_threads(), omp_get_thread_num());
    printf("\ta_shared \t= %d\n", a_shared);
    printf("\tb_private \t= %d\n", b_private);
    printf("\tc_firstprivate \t= %d\n", c_firstprivate);
    printf("\te_atomic \t= %d\n", e_atomic);
    #pragma omp parallel default(none) shared(a_shared, e_atomic)
    private(b_private) firstprivate(c_firstprivate )
    {
```

```

int i;
int d_local_private;
d_local_private = a_shared + c_firstprivate;
f_threadprivate = omp_get_thread_num();
#pragma omp barrier
#pragma omp critical
{
    for(i=0;i<10;i++){
        a_shared ++;
    }
}
for(i=0;i<10;i++){
    c_firstprivate += omp_get_thread_num();
}
for(i=0;i<10;i++){
    #pragma omp atomic
    e_atomic+=omp_get_thread_num();
}
#pragma omp barrier
#pragma omp critical
{
    printf("\nw obszarze równoległym: aktualna liczba watkow %d, moj ID
%d\n", omp_get_num_threads(), omp_get_thread_num());
    printf("\ta_shared \t= %d\n", a_shared);
    printf("\tb_private \t= %d\n", b_private);
    printf("\tc_firstprivate \t= %d\n", c_firstprivate);
    printf("\td_local_private = %d\n", d_local_private);
    printf("\te_atomic \t= %d\n", e_atomic);
    printf("\tf_threadprivate = %d\n", f_threadprivate);
}
}
#pragma omp parallel default(none)
{
    #pragma omp critical
    {
        printf("\nw drugim obszarze równoległym: aktualna liczba watkow %d, moj
ID %d\n",
        omp_get_num_threads(), omp_get_thread_num());
        printf("\tf_threadprivate = %d\n", f_threadprivate);
    }
}
printf("po zakonczeniu obszaru rownoleglego:\n");
printf("\ta_shared \t= %d\n", a_shared);
printf("\tb_private \t= %d\n", b_private);
printf("\tc_firstprivate \t= %d\n", c_firstprivate);
printf("\te_atomic \t= %d\n", e_atomic);
}

```

3. Testy i wyniki

- Przed poprawkami: Program wykazywał błędy w przypadku zmiennej współdzielonej `a_shared` i zmiennej `e_atomic`. Wyniki zmiennych różniły się przy różnych liczbach wątków oraz uruchomieniach programu.

- Po poprawkach: Program został uruchomiony z liczbą wątków wynoszącą 5. Wyniki dla zmiennych `a_shared` i `e_atomic` stały się poprawne. Wartość zmiennej `f_threadprivate` została poprawnie zachowana w drugim obszarze równoległym.
- Wyniki działania programu
 - Stan początkowy przed obszarem równoległym:

```
Kompilator rozpoznaje dyrektywy OpenMP
przed wejściem do obszaru równoległego - nr_threads 1, thread ID 0
a_shared      = 1
b_private     = 2
c_firstprivate = 3
e_atomic      = 5
```

- Wyniki w obszarze równoległym

```
w obszarze równoległym: aktualna liczba wątków 5, moj ID 3
a_shared      = 51
b_private     = 32694
c_firstprivate = 33
d_local_private = 4
e_atomic      = 105
f_threadprivate = 3

w obszarze równoległym: aktualna liczba wątków 5, moj ID 1
a_shared      = 51
b_private     = 32694
c_firstprivate = 13
d_local_private = 4
e_atomic      = 105
f_threadprivate = 1

w obszarze równoległym: aktualna liczba wątków 5, moj ID 2
a_shared      = 51
b_private     = 32694
c_firstprivate = 23
d_local_private = 4
e_atomic      = 105
f_threadprivate = 2

w obszarze równoległym: aktualna liczba wątków 5, moj ID 0
a_shared      = 51
b_private     = 0
c_firstprivate = 3
d_local_private = 4
e_atomic      = 105
f_threadprivate = 0

w obszarze równoległym: aktualna liczba wątków 5, moj ID 4
a_shared      = 51
b_private     = 32694
c_firstprivate = 43
d_local_private = 4
e_atomic      = 105
f_threadprivate = 4
```

- Wyniki w drugim obszarze równoległym

```
w drugim obszarze równoległym: aktualna liczba wątków 5, moj ID 0  
f_threadprivate = 0  
  
w drugim obszarze równoległym: aktualna liczba wątków 5, moj ID 3  
f_threadprivate = 3  
  
w drugim obszarze równoległym: aktualna liczba wątków 5, moj ID 1  
f_threadprivate = 1  
  
w drugim obszarze równoległym: aktualna liczba wątków 5, moj ID 2  
f_threadprivate = 2  
  
w drugim obszarze równoległym: aktualna liczba wątków 5, moj ID 4  
f_threadprivate = 4
```

- Po zakończeniu obszaru równoległego

```
po zakończeniu obszaru równoległego:  
a_shared      = 51  
b_private     = 2  
c_firstprivate = 3  
e_atomic      = 105
```

4. Wnioski

- Wprowadzenie barier (`#pragma omp barrier`) zagwarantowało, że wszystkie wątki kończą swoje operacje w odpowiednim momencie. Brak synchronizacji mógłby powodować, że niektóre wątki przetwarzałyby dane przed zakończeniem pracy przez inne wątki, co prowadziłoby do błędów.
- Sekcje krytyczne (`#pragma omp critical`) zapobiegły konfliktom podczas modyfikacji zmiennej `a_shared`. Mechanizm ten zapewnił, że tylko jeden wątek mógł jednocześnie modyfikować tę zmienną.
- Użycie dyrektywy `#pragma omp atomic` dla zmiennej `e_atomic` pozwoliło na bezpieczne modyfikowanie jej przez wiele wątków. Operacje atomowe okazały się bardziej wydajne niż sekcje krytyczne w przypadku prostych operacji.

- Mechanizm `#pragma omp threadprivate` pozwolił każdemu wątkowi na przechowywanie prywatnej wartości zmiennej `f_threadprivate` pomiędzy obszarami równoległymi.
- Podział zmiennych na `shared`, `private` i `firstprivate` umożliwił kontrolowanie dostępu wątków do zasobów. Poprawne zarządzanie tymi atrybutami pozwoliło uniknąć kolizji oraz zapewniło spójność wyników.

pde

1. Przygotowanie środowiska roboczego

- Utworzono katalog o nazwie `lab_10/pde`. Do tego katalogu skopiowano plik `openmp_zaleznosci.c`.

2. Kompilacja programu

- Program został skompilowany z użyciem opcji:

`gcc -fopenmp openmp_zaleznosci.c -o openmp_zaleznosci -lm`

- Kompilacja z flagą `-fopenmp` umożliwia wykorzystanie OpenMP do zrównoleglenia programu, a `-lm` dołącza bibliotekę matematyczną, która jest wymagana dla funkcji `sin()`.

3. Uruchomienie programu i analiza kodu

Po pierwszym uruchomieniu programu (wersja sekwencyjna) zidentyfikowano, że w pętli występuje zależność danych. Zależność ta polega na tym, że wartość `A[i]` zależy od `A[i+2]`, co uniemożliwia poprawne zrównoleglenie tej pętli bez modyfikacji kodu. W szczególności problemem jest zapis do `A[i]` przy jednoczesnym odczycie z `A[i+2]` w tej samej iteracji.

4. Zrównoleglenie kodu

- W celu eliminacji zależności danych wprowadzono dodatkową tablicę C, która pozwala oddzielić odczyt od zapisu w pętli.

```
double* C = malloc((N+2)*sizeof(double));
```

```
for(i=0; i<N+2; i++) C[i] = (double)i/N;
```

- Zrównoleglenie kodu z użyciem OpenMP zostało zrealizowane następująco:

```
#pragma omp parallel for default(none) shared(A, B, C) num_threads(2)
for(i=0; i<N; i++){
    A[i] += C[i+2] + sin(B[i]);
}
```

- Wprowadzona tablica C przechowuje kopię danych z tablicy A. Dzięki temu eliminujemy zależność w pętli, gdyż zapis do A[i] nie wpływa na dane używane w innych iteracjach.
- Dyrektywa #pragma omp parallel for umożliwia równoległe wykonywanie iteracji pętli.
- Słowo kluczowe shared pozwala współdzielić tablice A, B, i C między wątkami.
- Parametr num_threads(2) ustala liczbę wątków na 2.

5. Uruchomienie programu i wyniki

- Program został uruchomiony w wersji sekwencyjnej i równoległej, a uzyskane wyniki przedstawiono poniżej:

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_10/pde$ ./openmp_zaleznosci
suma 1459701.114868, czas obliczen 0.006287
suma 1459701.114868, czas obliczen rownoleglych 0.003813
```

6. Analiza wyników:

Obie wersje programu zwracają tę samą wartość sumy (1459701.114868), co potwierdza poprawność równoległego wykonania.

Czas obliczeń wersji równoległej jest niemal dwukrotnie krótszy od czasu wersji sekwencyjnej (przy użyciu 2 wątków), co wskazuje na efektywne zrównoleglenie.

7. Wnioski

- Poprawne zrównoleglenie kodu wymagało wprowadzenia dodatkowej tablicy C, która pozwoliła wyeliminować zależności danych w pętli.
- OpenMP umożliwia łatwe zrównoleglenie kodu przy minimalnych zmianach, jednak wymaga dokładnej analizy zależności, by uniknąć błędów w obliczeniach.
- Wyniki testów pokazują, że wersja równoległa może znacznie przyspieszyć obliczenia na dużych zbiorach danych, co jest szczególnie istotne w aplikacjach wymagających dużej mocy obliczeniowej.